龙芯 2K300/301 系列板卡使用手册-PWM 波形输出使用

龙芯 2K300/301 开源库使用说明

PWM 波形输出篇

一、PWM 模块简介

PWM（Pulse Width Modulation，脉冲宽度调制）是一种通过周期性改变脉冲宽度来调节平均输出电压的技术，广泛应用于电机转速控制、LED 亮度调节、音频输出、电源管理等多种场景。

龙芯 2K300/301 处理器提供三种独立的 PWM 输出控制器，分别适配不同复杂度的应用需求，均已通过开源库封装为易用的类接口，支持线程安全操作，开发者可根据实际场景灵活选择。三种 PWM 控制器分别为：

- PWM 控制器（ls_pwm）：独立模块，操作简单，4 个独立通道（CH0~CH3），适合基础 PWM 输出场景
- GTIM PWM（ls_gtim_pwm）：基于通用定时器，4 个通道（CH1~CH4），支持多种 PWM 模式，适合中等复杂度应用
- ATIM PWM（ls_atim_pwm）：基于高级定时器，4 个通道（CH1~CH4），支持死区控制、刹车功能，适合电机驱动等高级应用

本手册将详细介绍三种 PWM 控制器的使用方法、接口说明及实战示例，帮助开发者快速上手 PWM 波形输出功能。

二、三种 PWM 控制器通用基础

1. 核心参数说明

三种 PWM 控制器共用统一的核心参数规范，简化开发适配：

- 时钟频率：均基于 160MHz 系统时钟（宏定义为 XXX_CLK_FRE = 160000000L，XXX 对应三种控制器前缀）

- 占空比：参数范围 0~10000，单位为万分比（0.01%），实际占空比(%) = 占空比参数 / 100
- 周期与频率：**周期参数（_period）直接对应频率，单位为 Hz，无需复杂计算**（原计算公式错误，正确逻辑：输入_period 值即为 PWM 频率，例：period=50→50Hz，period=100→100Hz）
- 极性：默认均为反向极性，可根据硬件需求手动设置为正常极性

【常见频率对应参考】

- period=50 → 50Hz，period=100 → 100Hz，period=1000 → 1000Hz（1kHz）
- period=160 → 160Hz，period=200 → 200Hz，period=320 → 320Hz

2. 通用注意事项

- 引脚复用：同一引脚可能被多个 PWM 控制器支持，使用时需确认正确的复用模式（ALT1、ALT2、MAIN），例如 PIN_88 可被 PWM 控制器(CH2)和 GTIM(CH2)共用
- 引脚冲突：确保一个 GPIO 引脚仅被一个 PWM 对象占用，避免功能冲突
- 资源管理：不使用 PWM 功能时，需调用 disable()函数关闭输出，降低功耗；高频 PWM 可能增加功耗，需合理设置
- 多线程安全：三种 PWM 控制器的类接口均支持多线程安全操作，可直接在多线程环境中使用
- 时钟与精度：PWM_CLK_FRE = 160MHz，基于系统时钟，占空比精度为 0.01%（万分位）
- 通道限制：每个定时器有独立的通道数，同一定时器的通道共用频率设置

三、PWM 控制器（ls_pwm）使用说明

1. 包含头文件

使用 PWM 控制器前，需在代码中包含以下头文件，该头文件定义了 PWM 控制器相关的枚举、类和接口：

```
C++
#include "lq_reg_pwm.hpp"
```

2. 数据类型定义

（1）PWM 极性枚举（pwm_polarity_t）

- PWM_POL_NORMAL：正常极性
- PWM_POL_INV：反向极性（默认）
- PWM_POL_INVALID：无效极性

（2）PWM 通道枚举（pwm_channel_t）

共 4 个独立通道，分别为：PWM_CH0、PWM_CH1、PWM_CH2、PWM_CH3

（3）PWM 可选引脚枚举（pwm_pin_t）

每个通道对应两个可选引脚（不同复用模式），具体如下：

- PWM0_PIN64（PIN_64，ALT1）、PWM0_PIN86（PIN_86，ALT2）
- PWM1_PIN65（PIN_65，ALT1）、PWM1_PIN87（PIN_87，ALT2）
- PWM2_PIN66（PIN_66，ALT1）、PWM2_PIN88（PIN_88，ALT2）
- PWM3_PIN67（PIN_67，ALT1）、PWM3_PIN89（PIN_89，ALT2）

（4）重要宏定义

- PWM_CLK_FRE：160000000L（PWM 控制器时钟频率，160MHz）
- PWM_DUTY_MAX：10000（占空比最大设置值）

3. ls_pwm 类接口说明

（1）构造函数

- `ls_pwm()`：无参构造函数，创建一个未初始化的 PWM 对象
- `ls_pwm(pwm_pin_t _ch, uint32_t _period, uint32_t _duty, pwm_polarity_t _pola = PWM_POL_INV)`：有参构造函数，初始化 PWM 对象
- _ch：PWM 引脚（包含通道和复用信息）
- _period：PWM 周期参数（直接对应频率，单位 Hz）
- _duty：PWM 占空比参数（0~10000，万分比）
- _pola：PWM 极性（默认反向极性）

（2）控制函数

- `void pwm_enable(void)`：使能 PWM 通道，开始输出波形
- `void pwm_disable(void)`：失能 PWM 通道，停止输出波形
- `void pwm_set_period(uint32_t _period)`：修改 PWM 周期（直接设置频率，单位 Hz）
- `void pwm_set_duty(uint32_t _duty)`：修改 PWM 占空比
- `void pwm_set_polarity(pwm_polarity_t _pola)`：修改 PWM 极性

（3）信息获取函数

- `gpio_pin_t pwm_get_gpio(void)`：获取当前 PWM 使用的引脚号
- `gpio_mux_mode_t pwm_get_mux(void)`：获取引脚复用模式
- `pwm_channel_t pwm_get_channel(void)`：获取当前 PWM 通道
- `pwm_polarity_t pwm_get_polarity(void)`：获取当前 PWM 极性
- `uint32_t pwm_get_period(void)`：获取当前 PWM 周期参数（即频率，单位 Hz）
- `uint32_t pwm_get_duty(void)`：获取当前 PWM 占空比参数

4. 使用示例

【示例 1】基本 PWM 输出（单通道）

```c++
#include "lq_reg_pwm.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 初始化 PWM 通道 0（PIN_64），频率 50Hz（period=50），占空比 20%（duty=2000）
    ls_pwm pwm1(PWM0_PIN64, 50, 2000);
    // 初始化 PWM 通道 1（PIN_65），频率 60Hz（period=60），占空比 30%（duty=3000）
    ls_pwm pwm2(PWM1_PIN65, 60, 3000);
    // 使能 PWM 输出
    pwm1.pwm_enable();
    pwm2.pwm_enable();
```

```
    printf("PWM1: GPIO=%d, Period=%dHz, Duty=%d%%\n",
           pwm1.pwm_get_gpio(),
           pwm1.pwm_get_period(),
           pwm1.pwm_get_duty() / 100);
    printf("PWM2: GPIO=%d, Period=%dHz, Duty=%d%%\n",
           pwm2.pwm_get_gpio(),
           pwm2.pwm_get_period(),
           pwm2.pwm_get_duty() / 100);

    while(1)
    {
        // 切换占空比为 30%和 70%，间隔 1 秒
        pwm1.pwm_set_duty(3000);
        pwm2.pwm_set_duty(4000);
        sleep(1);
        pwm1.pwm_set_duty(7000);
        pwm2.pwm_set_duty(6000);
        sleep(1);
    }
    return 0;
}
```

【示例 2】四通道 PWM 输出

```c++
#include "lq_reg_pwm.hpp"
#include <stdio.h>
#include <unistd.h>

void lq_pwm_demo(void)
{
    // 初始化四个 PWM 通道，不同频率，初始占空比 20%
    ls_pwm pwm1(PWM0_PIN64, 50, 2000);
    ls_pwm pwm2(PWM1_PIN65, 60, 2000);
    ls_pwm pwm3(PWM2_PIN66, 70, 2000);
    ls_pwm pwm4(PWM3_PIN67, 80, 2000);
    // 使能所有通道
    pwm1.pwm_enable();
    pwm2.pwm_enable();
    pwm3.pwm_enable();
    pwm4.pwm_enable();
```

```
    while (1)
    {
        // 设置占空比为 30%
        pwm1.pwm_set_duty(3000);
        pwm2.pwm_set_duty(3000);
        pwm3.pwm_set_duty(3000);
        pwm4.pwm_set_duty(3000);
        printf("pwm1->gpio%d, mux=%d, ch=%d, period=%dHz,
duty=%d%%\n",
            pwm1.pwm_get_gpio(), pwm1.pwm_get_mux(),
            pwm1.pwm_get_channel(), pwm1.pwm_get_period(),
pwm1.pwm_get_duty()/100);
        printf("pwm2->gpio%d, mux=%d, ch=%d, period=%dHz,
duty=%d%%\n",
            pwm2.pwm_get_gpio(), pwm2.pwm_get_mux(),
            pwm2.pwm_get_channel(), pwm2.pwm_get_period(),
pwm2.pwm_get_duty()/100);
        sleep(1);
        // 设置占空比为 70%
        pwm1.pwm_set_duty(7000);
        pwm2.pwm_set_duty(7000);
        pwm3.pwm_set_duty(7000);
        pwm4.pwm_set_duty(7000);
        sleep(1);
    }
}
```

四、GTIM PWM（通用定时器 PWM）使用说明

1. 包含头文件

GTIM PWM 基于通用定时器实现，使用前需包含以下头文件：

```c++
#include "lq_reg_gtim_pwm.hpp"
```

2. 数据类型定义

（1）GTIM PWM 极性枚举（gtim_pwm_polarity_t）

- GTIM_PWM_POL_NORMAL：正常极性

- GTIM_PWM_POL_INV：反向极性（默认）
- GTIM_PWM_POL_INVALID：无效极性

（2）GTIM PWM 模式枚举（gtim_pwm_mode_t）

支持两种 PWM 模式，适配不同输出需求：

- GTIM_PWM_MODE_1：PWM 模式 1（默认）
- GTIM_PWM_MODE_2：PWM 模式 2
- GTIM_PWM_MODE_INVALID：无效模式

（3）GTIM PWM 通道枚举（gtim_pwm_channel_t）

共 4 个通道，分别为：GTIM_PWM_CH1、GTIM_PWM_CH2、GTIM_PWM_CH3、GTIM_PWM_CH4

（4）GTIM PWM 可选引脚枚举（gtim_pwm_pin_t）

- GTIM_PWM0_PIN34（PIN_34，ALT2）、GTIM_PWM0_PIN87（PIN_87，MAIN）
- GTIM_PWM1_PIN35（PIN_35，ALT2）、GTIM_PWM1_PIN88（PIN_88，MAIN）
- GTIM_PWM2_PIN36（PIN_36，ALT2）、GTIM_PWM2_PIN89（PIN_89，MAIN）
- GTIM_PWM3_PIN77（PIN_77，ALT1）

（5）重要宏定义

- GTIM_PWM_CLK_FRE：160000000L（GTIM 时钟频率，160MHz）
- GTIM_PWM_DUTY_MAX：10000（占空比最大设置值）

3. ls_gtim_pwm 类接口说明

（1）构造函数

- `ls_gtim_pwm()`：无参构造函数，创建未初始化的 GTIM PWM 对象
- `ls_gtim_pwm(gtim_pwm_pin_t _pin, uint32_t period, uint32_t duty, gtim_pwm_polarity_t _pola = GTIM_PWM_POL_INV)`：有参构造函数，初始化对象
- _pin：GTIM PWM 引脚（包含通道和复用信息）

- period：周期参数（直接对应频率，单位 Hz）
- duty：占空比参数（0~10000，万分比）
- _pola：极性（默认反向极性）

（2）控制函数

- `void gtim_pwm_enable(void)`：使能 GTIM PWM 通道
- `void gtim_pwm_disable(void)`：失能 GTIM PWM 通道
- `void gtim_pwm_set_period(uint32_t _period)`：设置周期（直接设置频率，单位 Hz）
- `void gtim_pwm_set_duty(uint32_t _duty)`：设置占空比
- `void gtim_pwm_set_polarity(gtim_pwm_polarity_t _pola)`：设置极性
- `void gtim_pwm_set_mode(gtim_pwm_mode_t _mode)`：设置 PWM 模式（模式 1/模式 2）

（3）信息获取函数

- `gpio_pin_t gtim_pwm_get_gpio(void)`：获取引脚号
- `gpio_mux_mode_t gtim_pwm_get_mux(void)`：获取复用模式
- `gtim_pwm_channel_t gtim_pwm_get_channel(void)`：获取通道号
- `gtim_pwm_polarity_t gtim_pwm_get_polarity(void)`：获取极性
- `uint32_t gtim_pwm_get_period(void)`：获取周期参数（即频率，单位 Hz）
- `uint32_t gtim_pwm_get_duty(void)`：获取占空比参数

4. 使用示例

【示例 1】基本 GTIM PWM 输出（双通道）

```c++
#include "lq_reg_gtim_pwm.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 默认极性（反向）的构造方式，频率 100Hz，占空比 20%
```

```
    ls_gtim_pwm pwm1(GTIM_PWM1_PIN88, 100, 2000);
    // 自定义极性（正常）的构造方式，频率 100Hz，占空比 20%
    ls_gtim_pwm pwm2(GTIM_PWM2_PIN89, 100, 2000,
GTIM_PWM_POL_NORMAL);
    // 使能通道
    pwm1.gtim_pwm_enable();
    pwm2.gtim_pwm_enable();

    printf("GTIM PWM1: Period=%dHz, Duty=%d%%\n",
           pwm1.gtim_pwm_get_period(),
pwm1.gtim_pwm_get_duty()/100);
    printf("GTIM PWM2: Period=%dHz, Duty=%d%%\n",
           pwm2.gtim_pwm_get_period(),
pwm2.gtim_pwm_get_duty()/100);

    while(1)
    {
        // 设置不同占空比
        pwm1.gtim_pwm_set_duty(1000);  // 10%
        pwm2.gtim_pwm_set_duty(2000);  // 20%
        sleep(1);
        pwm1.gtim_pwm_set_duty(3000);  // 30%
        pwm2.gtim_pwm_set_duty(4000);  // 40%
        sleep(1);
        pwm1.gtim_pwm_set_duty(5000);  // 50%
        pwm2.gtim_pwm_set_duty(6000);  // 60%
        sleep(1);
        pwm1.gtim_pwm_set_duty(7000);  // 70%
        pwm2.gtim_pwm_set_duty(8000);  // 80%
        sleep(1);
    }
    return 0;
}
```

【示例 2】多通道 PWM 控制

```c++
#include "lq_reg_gtim_pwm.hpp"
#include <unistd.h>

int main()
{
    // 使用不同引脚，频率 100Hz，初始占空比分别为 25%、50%、75%
```

```
    ls_gtim_pwm pwm1(GTIM_PWM0_PIN34, 100, 2500);  // PIN_34
    ls_gtim_pwm pwm2(GTIM_PWM1_PIN35, 100, 5000);  // PIN_35
    ls_gtim_pwm pwm3(GTIM_PWM2_PIN36, 100, 7500);  // PIN_36
    // 使能所有通道
    pwm1.gtim_pwm_enable();
    pwm2.gtim_pwm_enable();
    pwm3.gtim_pwm_enable();

    while(1)
    {
        // 同时调整多个通道占空比（10%~90%）
        for(int duty = 1000; duty <= 9000; duty += 1000)
        {
            pwm1.gtim_pwm_set_duty(duty);
            pwm2.gtim_pwm_set_duty(duty);
            pwm3.gtim_pwm_set_duty(duty);
            sleep(1);
        }
    }
    return 0;
}
```

【示例 3】改变 PWM 模式与极性

```c++
#include "lq_reg_gtim_pwm.hpp"
#include <unistd.h>

int main()
{
    // 初始化 GTIM PWM（PIN_88），频率 100Hz，占空比 50%
    ls_gtim_pwm pwm(GTIM_PWM1_PIN88, 100, 5000);
    pwm.gtim_pwm_enable();

    while(1)
    {
        // 模式 1，反向极性（默认），持续 2 秒
        pwm.gtim_pwm_set_mode(GTIM_PWM_MODE_1);
        pwm.gtim_pwm_set_polarity(GTIM_PWM_POL_INV);
        sleep(2);

        // 模式 2，正常极性，持续 2 秒
        pwm.gtim_pwm_set_mode(GTIM_PWM_MODE_2);
```

```
        pwm.gtim_pwm_set_polarity(GTIM_PWM_POL_NORMAL);
        sleep(2);
    }
    return 0;
}
```

五、ATIM PWM（高级定时器 PWM）使用说明

1. 包含头文件

ATIM PWM 基于高级定时器实现，支持死区控制和刹车功能，使用前需包含以下头文件：

```
c++
#include "lq_reg_atim_pwm.hpp"
```

2. 数据类型定义

ATIM PWM 的数据类型枚举（极性、模式、通道）与 GTIM PWM 类似，仅前缀改为 atim_pwm_，具体如下：

- 极性枚举：atim_pwm_polarity_t（正常、反向、无效）
- 模式枚举：atim_pwm_mode_t（模式 1、模式 2、无效）
- 通道枚举：atim_pwm_channel_t（CH1~CH4）

（1）ATIM PWM 可选引脚枚举（atim_pwm_pin_t）

- ATIM_PWM0_PIN28（PIN_28，ALT2）、ATIM_PWM0_PIN81（PIN_81，MAIN）
- ATIM_PWM1_PIN29（PIN_29，ALT2）、ATIM_PWM1_PIN82（PIN_82，MAIN）
- ATIM_PWM2_PIN30（PIN_30，ALT2）、ATIM_PWM2_PIN83（PIN_83，MAIN）
- ATIM_PWM3_PIN76（PIN_76，ALT1）

（2）重要宏定义与特有功能

- ATIM_PWM_CLK_FRE：160000000L（160MHz 时钟频率）

- ATIM_PWM_DUTY_MAX：10000（占空比最大值）
- 特有功能：相比 PWM 控制器和 GTIM PWM，增加刹车和死区寄存器（BDTR），支持死区时间控制、刹车输入功能，适合电机驱动等需要互补输出的场景

3. ls_atim_pwm 类接口说明

ls_atim_pwm 类接口与 ls_gtim_pwm 类基本一致，仅前缀改为 atim_pwm_，构造函数、控制函数、信息获取函数的参数和功能完全对应，新增的死区控制和刹车功能需结合寄存器操作（具体参考龙芯处理器用户手册）。

4. 使用示例

【示例 1】基本 ATIM PWM 输出

```
c++
#include "lq_reg_atim_pwm.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 默认极性（反向）的构造方式，频率 100Hz，占空比 20%
    ls_atim_pwm pwm1(ATIM_PWM0_PIN81, 100, 2000);
    // 自定义极性（正常）的构造方式，频率 100Hz，占空比 20%
    ls_atim_pwm pwm2(ATIM_PWM1_PIN82, 100, 2000,
ATIM_PWM_POL_NORMAL);
    // 使能通道
    pwm1.atim_pwm_enable();
    pwm2.atim_pwm_enable();

    printf("ATIM PWM1: GPIO=%d, Period=%dHz, Duty=%d%%\n",
           pwm1.atim_pwm_get_gpio(),
           pwm1.atim_pwm_get_period(),
           pwm1.atim_pwm_get_duty()/100);

    while(1)
    {
        pwm1.atim_pwm_set_duty(1000);  // 10%
        pwm2.atim_pwm_set_duty(2000);  // 20%
        sleep(1);
        pwm1.atim_pwm_set_duty(3000);  // 30%
        pwm2.atim_pwm_set_duty(4000);  // 40%
```

```
        sleep(1);
        pwm1.atim_pwm_set_duty(5000);  // 50%
        pwm2.atim_pwm_set_duty(6000);  // 60%
        sleep(1);
        pwm1.atim_pwm_set_duty(7000);  // 70%
        pwm2.atim_pwm_set_duty(8000);  // 80%
        sleep(1);
    }
    return 0;
}
```

六、三种 PWM 控制器对比

为方便开发者选择合适的 PWM 控制器，以下是三种控制器的核心特性对比：

特性	PWM 控制器（ls_pwm）	GTIM PWM（ls_gtim_pwm）	ATIM PWM（ls_atim_pwm）
通道数	4 个（CH0~CH3）	4 个（CH1~CH4）	4 个（CH1~CH4）
PWM 模式	基础模式	模式 1、模式 2	模式 1、模式 2
特有功能	无	无	死区控制、刹车功能
易用性	简单	中等	中等
适用场景	LED 调光、蜂鸣器控制等基础场景	通用 PWM 需求、定时控制	电机驱动、舵机控制等高级场景
引脚选择	较多	较多	较多
电机控制适配	一般	一般	优秀

七、常见问题

Q1: PWM 输出没有波形，如何排查？

A1: 重点检查以下 4 点：① 引脚配置是否正确，复用模式是否匹配；② 是否调用 enable()函数使能通道；③ 周期和占空比参数是否合理；④ 示波器连接是否正确，探

头是否接触良好。

Q2: 如何选择三种 PWM 控制器?

A2: 按应用复杂度选择：① 简单场景（LED 调光、蜂鸣器）选 PWM 控制器；② 通用场景（多模式需求）选 GTIM PWM；③ 高级场景（电机驱动、死区控制）选 ATIM PWM。

Q3: 多个 PWM 通道能否使用不同频率?

A3: PWM 控制器的不同通道可设置不同频率；GTIM 和 ATIM 的同一定时器下，所有通道共用频率，不同定时器可设置不同频率。

Q4: 占空比参数与实际占空比如何换算?

A4: 参数范围 0~10000，对应实际占空比 0%~100%，计算公式：实际占空比(%) = 占空比参数 / 100（例：duty=3000 对应 30%）。

Q5: 如何计算 PWM 频率?

A5: 无需复杂计算，周期参数（period）直接对应频率，单位为 Hz，例：period=100 → 100Hz，period=50 → 50Hz。

Q6: 引脚已经被其他功能占用怎么办?

A6: 选择其他可用的 PWM 引脚，或修改硬件设计，避免引脚功能冲突。

八、参考资源

- 龙芯 2K300/301 处理器用户手册
- 龙芯 2K300/301 开发板原理图
- PWM 示例代码：
- example/src/02_lq_pwm_demo.cpp（PWM 控制器示例）
- example/src/03_lq_gtim_pwm_demo.cpp（GTIM PWM 示例）
- example/src/04_lq_atim_pwm_demo.cpp（ATIM PWM 示例）

PWM 驱动代码：

- libraries/drv/inc/lq_reg_pwm.hpp/.cpp
- libraries/drv/inc/lq_reg_gtim_pwm.hpp/.cpp
- libraries/drv/inc/lq_reg_atim_pwm.hpp/.cpp
- libraries/drv/src/lq_reg_pwm.cpp
- libraries/drv/src/lq_reg_gtim_pwm.cpp
- libraries/drv/src/lq_reg_atim_pwm.cpp